

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA



PLAN DE PRUEBA SISTEMA DE SOLICITUDES

Diego Iván Correa Navarrete
Juan Antonio González del Río
Diego Arturo Ramos Ávila
Ángel David Carlos Silva

LICENCIATURA EN INGENIERÍA DE SOFTWARE

Junio de 2026

Índice general

Abstract	2
Historial de Versiones	3
Introducción	4
1. Estrategia de Test	5
1.1. Alcance del Testing	5
1.1.1. Características a ser testeadas	5
1.1.2. Características que NO serán testeadas	7
1.2. Tipo de Test	7
1.3. Riesgos y Problemas	8
1.4. Logística de Pruebas	9
1.4.1. ¿Quiénes realizarán las pruebas?	9
1.4.2. ¿Cuándo ocurrirán las pruebas?	9
2. Objetivos del Test	10
3. Criterio de Prueba	11
3.1. Criterio de Suspensión	11
3.2. Criterio de Salida	11
4. Planeación de Recursos	12
4.1. Recursos del sistema	12
4.2. Recursos Humanos	13
5. Ambiente de Pruebas	14
6. Calendario y Estimación	15
7. Entregables de Testing	16

Abstract

Este documento provee una vista general del proyecto **Sistema de Solicitudes** de la Universidad Autónoma de Zacatecas y describe la estrategia de pruebas del producto, la lista de los entregables de *testing* y el plan para llevar a cabo dichas actividades durante el desarrollo.

El Sistema de Solicitudes es una aplicación web monolítica construida con Django 5 que digitaliza el ciclo de vida de las solicitudes académicas y administrativas de la institución: catálogo dinámico de tipos de solicitud, formularios configurables, generación de documentos PDF a partir de plantillas, máquina de estados con enrutamiento por roles, gestión de archivos adjuntos, notificaciones por correo y administración de mentores. El presente plan prescribe el alcance, el enfoque, los recursos y el calendario de todas las actividades de prueba asociadas, así como los criterios que determinan su terminación exitosa.

Historial de Versiones

uazblue!12 Versión	Fecha	Autor	Descripción
v1.0	Junio 2026	Equipo de desarrollo (Correa, González, Ramos, Carlos)	Versión inicial del Plan de Prueba del Sistema de Solicitudes. Se define la estrategia de pruebas unitarias, de integración, funcionales/de aceptación y de performance, los criterios de salida y los entregables de <i>testing</i> .

Introducción

El presente Plan de Prueba está diseñado para prescribir el **alcance**, el **enfoque**, los **recursos** y el **calendario** de todas las actividades de *testing* del proyecto Sistema de Solicitudes de la Universidad Autónoma de Zacatecas.

En el plan se identifican los elementos y características a ser probados, los tipos de prueba a ejecutar, el personal responsable de las pruebas, los recursos de hardware y software requeridos, el calendario estimado para completar el *testing* y los riesgos asociados con su ejecución. El documento sirve como guía operativa para el equipo y como referencia de cumplimiento frente a la especificación de requerimientos de software (SRS) y al diseño del sistema (SDD).

El enfoque de pruebas se apoya en la metodología **TDD** (Test-Driven Development): las pruebas se escriben antes que el código de producción, de modo que cada requerimiento funcional queda respaldado por una o más pruebas ejecutables. El *stack* de pruebas se compone de **pytest** con **pytest-django** para las pruebas unitarias y de integración, **behave** con **Selenium** para las pruebas funcionales/de aceptación en navegador, **JMeter** para las pruebas de performance, **coverage** para la medición de cobertura, **radon** para la complejidad ciclomática y **ruff** para la conformidad con PEP 8.

Capítulo 1

Estrategia de Test

1.1. Alcance del Testing

1.1.1. Características a ser testeadas

Todas las características funcionales del Sistema de Solicitudes definidas en la especificación de requerimientos de software deben ser probadas. Estas características corresponden a los once requerimientos funcionales (RF-01 a RF-11) y a los casos de uso CU-01 a CU-17 del SRS, distribuidos entre los cinco roles del sistema: **ALUMNO**, **DOCENTE**, **CONTROL_ESCOLAR**, **RESPONSABLE_PROGRAMA** y **ADMIN**.

uazblue!12 Caso	Característica	Descripción de lo que se prueba
CU-01	Crear solicitud	El solicitante selecciona un tipo, llena el formulario dinámico, adjunta archivos, se genera el folio SOL-AAAA-NNNNN y la solicitud queda en estado “Creada”; se valida la regla de comprobante de pago y la exención de mentores.
CU-02	Consultar solicitudes propias	El solicitante lista sus solicitudes con folio, tipo, fecha y estado, y aplica filtros.
CU-03	Ver detalle de solicitud	Visualización del formulario, archivos adjuntos e historial de seguimiento.
CU-04	Cancelar solicitud propia	El solicitante cancela una solicitud que se encuentra en estado “Creada”.
CU-05	Ver seguimiento	Consulta del historial de cambios de estado con fechas y observaciones.
CU-06	Listar solicitudes asignadas	El personal ve únicamente las solicitudes dirigidas a su rol, con filtros y búsqueda.
CU-07	Atender solicitud	El personal revisa el detalle de una solicitud asignada y ejecuta las acciones de resolución.
CU-08	Cambiar estado	Transiciones de la máquina de estados (En proceso, Finalizada, Cancelada) con observaciones.
CU-09	Generar documento PDF	Generación del PDF a partir de la plantilla del tipo, sustituyendo variables con datos del solicitante.
CU-10	Gestionar tipos de solicitud	El administrador crea, edita o elimina tipos del catálogo.
CU-11	Gestionar formularios dinámicos	Configuración de los campos del formulario de cada tipo.
CU-12	Gestionar plantillas	Administración de plantillas de documento con variables sustituibles.
CU-13	Gestionar mentores	Alta/baja/consulta de mentores de forma manual o mediante carga CSV.
CU-14	Ver dashboard y métricas	Estadísticas de solicitudes por tipo, estado y período.
CU-15	Exportar reportes	Exportación de métricas en formato CSV o PDF.
CU-17	Notificar por correo	Despacho automático de correos al personal al crear una solicitud y al solicitante al cambiar el estado, incluyendo el <i>fallback</i> ante fallas de SMTP.

Adicionalmente se prueban las reglas transversales asociadas: el enrutamiento de solicitudes por rol responsable (RF-06), la fotografía (*snapshot*) de la definición de campos del

tipo al momento de la creación (RF-04), la generación secuencial y única del folio por año, la validación del tamaño máximo de archivo (10 MB) y de los tipos de archivo permitidos.

1.1.2. Características que NO serán testeadas

Las siguientes características quedan **fuera del alcance** de este plan, porque corresponden a sistemas de terceros sobre los que el equipo no tiene control, o porque no forman parte de la especificación de requerimientos de software del Sistema de Solicitudes. Su exclusión se justifica a continuación.

- **Proveedor de autenticación externo (validación de JWT de terceros / SIGA).** El sistema no implementa autenticación propia; recibe y confía en un JWT emitido por un proveedor externo (RNF-01) y consulta datos de usuario a SIGA (RNF-02). La emisión, firma y validación del token por parte del proveedor, así como la disponibilidad de SIGA, son responsabilidad de esos servicios y no pueden ser controlados ni reproducidos de forma fiable por el equipo. Sí se prueba, en cambio, el *middleware* que decodifica el token ya recibido y el *fallback* a los datos del JWT cuando SIGA no responde, empleando dobles de prueba.
- **Envío real a través del servidor SMTP externo.** El servidor de correo es un servicio de terceros (IE-03). Se prueba el *dispatch* de las notificaciones (que el sistema construye y entrega el mensaje al backend de correo) y el comportamiento de *fallback* ante una falla de SMTP (la operación de negocio no debe verse afectada), pero no se prueba la entrega efectiva del correo por parte del servidor de un tercero.
- **Interfaces de hardware.** El sistema es una aplicación web que no interactúa directamente con dispositivos de hardware; no existen controladores ni interfaces físicas que probar.

1.2. Tipo de Test

En el proyecto Sistema de Solicitudes se ejecutan **cuatro** tipos de prueba a lo largo del desarrollo:

- **(a) Pruebas Unitarias.** Verifican de forma aislada el comportamiento de las unidades de software más pequeñas (servicios, repositorios, formularios, esquemas Pydantic, funciones de dominio). Se desarrollan de manera colaborativa: cada integrante escribe las pruebas de los módulos a su cargo. Se implementan con `pytest` + `pytest-django` (sobre el estilo `TestCase` cuando aplica) y constituyen el grueso de la suite, con **769 pruebas (714 funciones de test únicas)**.
- **(b) Pruebas Funcionales / de Aceptación.** Simulan el comportamiento del sistema tal como lo experimenta el usuario final, para evaluar la conformidad con los requerimientos especificados y las historias de usuario. Se redactan en **Gherkin** y se ejecutan con **behave** y **Selenium**, controlando un navegador real. La suite de aceptación cubre **7 features** y **20 escenarios** que recorren los flujos principales de los cinco roles (creación, consulta, atención, cambio de estado y administración).

- **(c) Pruebas de Performance.** Evalúan el comportamiento del sistema bajo carga y concurrencia. Se ejecutan con **JMeter** sobre **2 casos**: (1) *login* con 10 usuarios concurrentes y (2) alta de solicitud con 5 usuarios concurrentes. Se observan tiempos de respuesta, tasa de error y *throughput*.
- **(d) Pruebas de Integración.** Verifican que los distintos componentes colaboren correctamente entre sí. En la arquitectura por capas de Django (View → Service → Repository sobre el ORM), las pruebas de integración están **entremezcladas con las unitarias**: al probar una vista a través del cliente de pruebas de Django se integran de hecho el servicio, el repositorio y el ORM contra una base de datos real, así como la serialización de DTOs Pydantic y el *rendering* de plantillas. Estas pruebas se identifican con el *marker integration* de **pytest**, lo que permite ejecutarlas o aislarlas de forma selectiva (**pytest -m integration**).

1.3. Riesgos y Problemas

uazblue!12 Riesgo	Mitigación
Dependencia del proveedor de autenticación externo (JWT) y de SIGA: una falla o indisponibilidad bloquearía las pruebas que requieren un usuario autenticado.	Aislar la dependencia mediante dobles de prueba (tokens fabricados, respuestas de SIGA simuladas con responses); probar el <i>fallback</i> a datos del JWT y no depender de los servicios reales en la suite automatizada.
Fallas del servidor SMTP externo durante las pruebas de notificación.	Usar el <i>backend</i> de correo en memoria de Django en pruebas; verificar únicamente el <i>dispatch</i> y el comportamiento de <i>fallback</i> , garantizando que la operación de negocio no se vea afectada.
Datos sembrados (<i>seed</i>) inconsistentes o insuficientes que invaliden los escenarios.	Construir datos con model_bakery y <i>fixtures</i> deterministas; congelar el tiempo con freezegun donde el folio o las fechas sean relevantes.
Tiempo limitado del equipo frente al volumen de casos a cubrir.	Priorizar por criticidad (flujos núcleo CU-01, CU-07, CU-08 primero); paralelizar el trabajo por módulo entre los integrantes; aprovechar TDD para evitar retrabajo.
Pruebas <i>flaky</i> (intermitentes) en navegador con Selenium por sincronización o tiempos de carga.	Emplear esperas explícitas, fijar la versión de Chrome/ <i>driver</i> , ejecutar en un entorno Docker reproducible y reintentar escenarios marcados como inestables antes de declararlos fallidos.

1.4. Logística de Pruebas

1.4.1. ¿Quiénes realizarán las pruebas?

Para este proyecto las pruebas serán realizadas por los propios miembros del equipo de desarrollo, quienes fungen con el rol de *testers* con el objetivo de optimizar los costos y los tiempos del proyecto. Cada integrante es responsable de escribir y mantener las pruebas del módulo a su cargo, además de participar en la construcción de la suite de aceptación compartida.

1.4.2. ¿Cuándo ocurrirán las pruebas?

Conforme a la metodología **TDD**, las pruebas se escriben **antes** que el código de producción y se ejecutan de forma continua durante todo el desarrollo. Los *testers* del equipo pueden comenzar la ejecución de las pruebas cuando las siguientes entradas se encuentren listas:

- Se cuenta con el ambiente de pruebas de Django configurado y reproducible (contenedores Docker con la base de datos de pruebas).
- De acuerdo con TDD, el desarrollo del software se realiza a la par de la escritura de las pruebas (*prueba primero*: RED → GREEN → REFACTOR).
- Se cuenta con la definición de los requerimientos técnicos y los criterios de aceptación de las historias de usuario del proyecto.
- Se dispone de los datos de prueba y *fixtures* necesarios para los escenarios.

Capítulo 2

Objetivos del Test

Los objetivos de la prueba son verificar la funcionalidad del Sistema de Solicitudes de la UAZ y, mediante la implementación de la metodología TDD, desarrollar dicha funcionalidad a partir de la creación de diferentes pruebas: pruebas unitarias, pruebas de integración, pruebas funcionales/de aceptación y pruebas de performance.

El proyecto se enfoca en probar la funcionalidad descrita en las historias de usuario y casos de uso del SRS, teniendo en cuenta el manejo de las operaciones principales del sistema:

- **Catálogo y formularios:** gestión de tipos de solicitud, formularios dinámicos y plantillas (CU-10, CU-11, CU-12).
- **Ciclo de vida de solicitudes:** creación con folio único, consulta, detalle, cancelación y seguimiento (CU-01 a CU-05).
- **Atención por el personal:** listado de asignadas, atención, cambio de estado y generación de PDF, con enrutamiento por rol (CU-06 a CU-09).
- **Administración:** mentores, dashboard de métricas y exportación de reportes (CU-13, CU-14, CU-15).
- **Notificaciones:** despacho de correos por evento y *fallback* ante fallas (CU-17).

El propósito es garantizar que estas operaciones se implementen correctamente y funcionen de manera estable en un entorno real de trabajo, respetando las reglas de negocio y la separación de responsabilidades por rol.

Capítulo 3

Criterio de Prueba

3.1. Criterio de Suspensión

En caso de que los miembros del equipo detecten que las pruebas no están contribuyendo positivamente al desarrollo del proyecto conforme a la metodología TDD —por ejemplo, ante una tasa elevada de fallos no atribuibles al código bajo prueba, dependencias externas inestables o un ambiente de pruebas roto— se suspenderá la ejecución, se revisarán los apartados descritos por TDD, se harán las correcciones necesarias en las pruebas o en el ambiente y se retomará el ciclo RED → GREEN → REFACTOR para continuar con el desarrollo del proyecto.

3.2. Criterio de Salida

Definen los criterios que denotan una terminación exitosa del desarrollo del proyecto con sus respectivas pruebas:

- De acuerdo con la metodología TDD, las pruebas definidas cubren el **100 %** de las funcionalidades requeridas del proyecto, con un objetivo de **cobertura de 100 %** medida con **coverage** sobre modelos, vistas, formularios y servicios (reporte de cobertura adjunto a los entregables).
- Las pruebas de aceptación con **behave** y **Selenium** cubren las historias de usuario descritas en el SRS; las pruebas unitarias se implementan con **TestCase/pytest**.
- Todos los métodos del sistema presentan una **complejidad ciclomática** ≤ 10 , verificada con **radon**.
- El código cumple con el estándar de estilo **PEP 8**, verificado con **ruff**.
- La totalidad de la suite automatizada (unitarias, integración y aceptación) pasa en verde en el ambiente de pruebas, y las pruebas de performance con JMeter se ejecutan dentro de umbrales aceptables de tiempo de respuesta y tasa de error.

Capítulo 4

Planeación de Recursos

4.1. Recursos del sistema

Recurso No.	Recurso	Descripción
1	Contenedores Docker	Orquestación del ambiente de desarrollo y pruebas (<i>web</i> + base de datos + MailHog) de forma reproducible en cada equipo.
2	PostgreSQL	Motor de base de datos para el ambiente de pruebas e integración; SQLite para ejecuciones locales rápidas.
3	Python 3.12+ y Django 5	Plataforma de ejecución del sistema y de la suite de pruebas (<code>pytest</code> , <code>pytest-django</code>).
4	Navegador Chrome + <i>driver</i>	Requerido por Selenium para las pruebas funcionales/de aceptación en navegador real.
5	Apache JMeter	Herramienta para las pruebas de performance (carga y concurrencia).
6	Herramientas de calidad	<code>coverage</code> (cobertura), <code>radon</code> (complejidad ciclomática) y <code>ruff</code> (PEP 8).
7	Equipos de cómputo	Una computadora por integrante con los requisitos mínimos para ejecutar Docker y el <i>stack</i> de pruebas.

4.2. Recursos Humanos

Integrante No.	Integrante	Rol de testing	Módulo a cargo
1	Diego Iván Correa Navarrete	Test Manager / Desarrollador en Test	Solicitudes: ciclo de vida y estados (CU-01, CU-04, CU-05, CU-08).
2	Juan Antonio González del Río	Desarrollador en Test	Catálogo, formularios y plantillas (CU-10, CU-11, CU-12).
3	Diego Arturo Ramos Ávila	Desarrollador en Test	Atención, generación de PDF y archivos adjuntos (CU-06, CU-07, CU-09).
4	Ángel David Carlos Silva	Desarrollador en Test	Mentores, reportes/dashboard y notificaciones (CU-13, CU-14, CU-15, CU-17).

Capítulo 5

Ambiente de Pruebas

Cada integrante del equipo cuenta con un equipo de cómputo con las características mínimas requeridas y con **Docker** instalado, lo que permite levantar de forma idéntica el ambiente de la aplicación (servicio *web* de Django, base de datos PostgreSQL y MailHog para inspección de correo) y ejecutar la suite de pruebas siguiendo la metodología TDD. El uso de contenedores garantiza que las pruebas se ejecuten sobre un entorno reproducible, eliminando las diferencias entre las máquinas de los desarrolladores.

Las pruebas unitarias, de integración y de aceptación se ejecutan de manera **local** dentro de los contenedores Docker de cada integrante. Para las pruebas de performance con JMeter se contempla, además, la disposición de un **servidor de pruebas / preproducción** que reproduzca condiciones más cercanas a producción, de modo que las mediciones de carga y concurrencia sean representativas y no se vean distorsionadas por los recursos de una máquina de desarrollo.

Capítulo 6

Calendario y Estimación

A continuación se presentan las tareas de prueba y su estimación aproximada, distribuidas entre abril y junio de 2026, en paralelo con el desarrollo incremental del sistema bajo TDD.

uazblue!12 Tarea	Responsables	Fecha aproximada
Definición de historias de usuario y criterios de aceptación	Todos	Abril 2026
Configuración del ambiente de pruebas (Docker, pytest, behave, Selenium)	Todos	Abril 2026
Elaboración del Plan de Prueba y plantilla de casos de prueba	Todos	Abril 2026
Pruebas unitarias y de integración por módulo (TDD)	Todos	Abril–Mayo 2026
Pruebas funcionales / de aceptación (behave + Selenium)	Todos	Mayo–Junio 2026
Pruebas de performance con JMeter	Todos	Junio 2026
Medición de cobertura, complejidad (radon) y estilo (ruff)	Todos	Junio 2026
Ejecución integral, reportes y matriz de trazabilidad	Todos	Junio 2026

Capítulo 7

Entregables de Testing

Como resultado de las actividades de prueba se entregan los siguientes artefactos:

- Código fuente del sistema acompañado de sus **pruebas unitarias** (suite `pytest` + `pytest-django`, 769 pruebas).
- **Pruebas de integración** (marcadas con el *marker integration*), entremezcladas con las unitarias.
- **Pruebas de aceptación** en `behave` + **Selenium** (7 *features*, 20 escenarios en Gherkin).
- **Matriz de trazabilidad** requerimientos/casos de uso ↔ pruebas.
- **Plantilla de casos de prueba** y los casos de prueba derivados de ella.
- El presente **Plan de Prueba**.
- **Pruebas de performance** con JMeter (2 casos) y sus resultados.
- **Reportes de cobertura** (coverage, objetivo 100 %) y de **complejidad ciclomática** (radon, métodos ≤ 10), junto con la verificación de estilo PEP 8 (`ruff`).
- Documentación de soporte: **SRS**, **SDD**, documento de **Visión y Alcance**, **esquema de la base de datos** y **manuales** (de usuario y técnico).